

EXP 6 : Write a Program to implement Sliding window protocol for Goback N.

Aim

To implement and simulate the Go-Back-N Automatic Repeat reQuest (ARQ) protocol using Python and study its operation under conditions of ACK loss.

Theory

Go-Back-N ARQ is a sliding-window error-control protocol used in the Data Link Layer for reliable data transmission. It provides both flow control and error control. In Go-Back-N ARQ, the sender can transmit multiple frames without waiting for an individual acknowledgement for every frame. The number of frames that can be sent at a time is determined by the window size.

The sender maintains two important values:

- **Base:** The sequence number of the oldest unacknowledged frame.
- **Next Sequence Number:** The sequence number of the next frame to be transmitted.

When a frame is successfully received, the receiver sends an ACK (Acknowledgement) to the sender. If an ACK is lost or a frame is not correctly received, the sender waits until a timeout occurs. When a timeout occurs, the sender goes back to the first unacknowledged frame and retransmits that frame and all subsequent frames that were already sent. This retransmission of multiple frames is the main feature of Go-Back-N ARQ.

Algorithm:

1. Sender Side Algorithm

Step 1: Initialize

- `base = 0`
- `next_seq = 0`
- Set `WINDOW_SIZE`
- Set `TOTAL_FRAMES`
- Define `TIMEOUT` value
- Initialize `acknowledged[]` as False for all frames

Step 2: Repeat until `base >= TOTAL_FRAMES`

a. Send Frames in Current Window

plaintext

CopyEdit

While `next_seq < base + WINDOW_SIZE` and `next_seq < TOTAL_FRAMES`:

 Send `frame[next_seq]`

 Start timer for `frame[next_seq]`

next_seq += 1

b. Wait for ACKs

plaintext

CopyEdit

For i = base to next_seq - 1:

 If ACK[i] is received before timeout:

 Mark acknowledged[i] = True

 base += 1

 Else:

 Timeout occurred:

 Print "Timeout for frame i"

 next_seq = base # Go back to base

 Break from ACK loop

c. Slide the Window

plaintext

CopyEdit

Window slides forward as ACKs are received

2. Receiver Side Algorithm

(Used in a simulation or real implementation)

Step 1: Initialize

- expected_frame = 0

Step 2: For each received frame:

plaintext

CopyEdit

If frame_number == expected_frame:

 Accept the frame

 Send ACK[expected_frame]

 expected_frame += 1

Else:

 Discard frame (out-of-order)

Send ACK[expected_frame - 1] again

3. ACK Loss Simulation (Optional)

- Introduce LOSS_PROBABILITY to simulate ACK loss:

plaintext

CopyEdit

If random() < LOSS_PROBABILITY:

Drop ACK (simulate lost ACK)

Else:

Send ACK to sender

Python Program:

```
import random
```

```
import time
```

```
# Configuration parameters
```

```
TOTAL_FRAMES = 10    # Total frames to send
```

```
WINDOW_SIZE = 4      # Sender's window size
```

```
LOSS_PROBABILITY = 0.2 # Probability of ACK loss
```

```
def send_frame(frame_num):
```

```
    print(f"Sender: Sending Frame {frame_num}")
```

```
    time.sleep(0.2) # Simulate delay
```

```
def receive_ack(frame_num):
```

```
    if random.random() < LOSS_PROBABILITY:
```

```
        print(f"Receiver: ACK {frame_num} lost!")
```

```
        return False
```

```
    else:
```

```
        print(f"Receiver: ACK {frame_num} received")
```

```
        return True
```

```

def go_back_n():
    base = 0
    next_seq = 0

    while base < TOTAL_FRAMES:
        # Send all frames in current window
        while next_seq < base + WINDOW_SIZE and next_seq < TOTAL_FRAMES:
            send_frame(next_seq)
            next_seq += 1

        # Wait for ACKs
        i = base
        while i < next_seq:
            print(f"Sender: Waiting for ACK {i}...")
            ack = receive_ack(i)
            if ack:
                base += 1
                i += 1
            else:
                print(f"Sender: Timeout! Resending from Frame {i}")
                next_seq = base # Go back and resend
                break

        print(f"Window: Base = {base}, Next Seq = {next_seq}")
        print("-" * 40)

if __name__ == "__main__":
    go_back_n()

```

Output: